Nested Queries in SQL

Week 11 · Lectures 21 & 22

A deep dive into **Simple Nested Queries** using the `WHERE` clause with `IN`, `NOT IN`, `ANY`, and `ALL` operators — essential tools for writing powerful, expressive SQL queries that compare values across tables and subqueries.

DATABASE SYSTEMS | SQL | ADVANCED QUERIES

What Is a Nested Query?

A **nested query** (also called a **subquery**) is a `SELECT` statement embedded within another SQL statement — typically inside the `WHERE` or `HAVING` clause of an outer query. The inner query executes first, and its result is used by the outer query to filter or compute results. Nested queries allow you to break complex problems into smaller, logical steps without requiring joins.

Structure of a Nested Query

The general form places a `SELECT` inside the `WHERE` clause:

```
SELECT column_list
FROM table1
WHERE column IN (
    SELECT column
    FROM table2
    WHERE condition
);
```

Key Characteristics

- The inner query is enclosed in parentheses
- Executed once per outer query row (correlated) or once total (non-correlated)
- Returns a single column of values for comparison
- Can be nested multiple levels deep
- Often more readable than equivalent joins
- Supported by all major SQL databases

The IN Operator

The `IN` operator tests whether a value matches **any value** returned by a subquery. It is one of the most commonly used operators in nested queries. The outer query returns rows where the tested column's value appears anywhere in the result set of the inner query.

Syntax

```
SELECT column_list
FROM table1
WHERE column_name IN (
    SELECT column_name
    FROM table2
    WHERE condition
);
```

How It Works

The subquery produces a list of values. The outer query checks each row: if the column value is found anywhere in that list, the row is included in the result. This is equivalent to multiple `OR` conditions but far more flexible when the list is dynamic.

Example: Find Employees in the 'Sales' Department

```
SELECT emp_name, salary
FROM Employee
WHERE dept_id IN (
    SELECT dept_id
    FROM Department
    WHERE dept_name = 'Sales'
);
```

The inner query returns all `dept_id` values for the Sales department. The outer query then returns all employees whose `dept_id` matches any of those values. If multiple departments matched, all their employees would be returned.

The NOT IN Operator

The `NOT IN` operator is the logical inverse of `IN`. It returns rows where the tested column's value does **not** appear in the result set of the subquery. This is useful for exclusion queries — finding records that have no matching counterpart in another table.

Syntax

```
SELECT column_list
FROM table1
WHERE column_name NOT IN (
    SELECT column_name
    FROM table2
    WHERE condition
);
```

⚠️ NULL Handling Warning

If the subquery returns even a single `NULL` value, `NOT IN` will return **no rows** at all, because `value NOT IN (..., NULL)` evaluates to `UNKNOWN` in SQL's three-valued logic. Always filter out NULLs in the subquery when using `NOT IN`.

Example: Find Customers Who Have Not Placed Orders

```
SELECT cust_name, email
FROM Customer
WHERE cust_id NOT IN (
    SELECT cust_id
    FROM Orders
    WHERE cust_id IS NOT NULL
);
```

The inner query returns all customer IDs that have placed at least one order. The outer query returns customers whose ID is **not** in that list — i.e., customers with no orders. Note the `IS NOT NULL` guard in the subquery to avoid the NULL pitfall.

IN vs. NOT IN: Side-by-Side Comparison

IN — Inclusion

Returns rows where the value **exists** in the subquery result.

- Used for "find matching records"
- Equivalent to `OR` across all subquery values
- Safe with NULLs in subquery
- Example: employees in a specific department

NOT IN — Exclusion

Returns rows where the value does **not exist** in the subquery result.

- Used for "find unmatched records"
- Equivalent to `AND NOT` across all subquery values
- **Dangerous** with NULLs — may return empty result
- Example: customers without any orders

Key Rule: Always add `WHERE column IS NOT NULL` inside the subquery when using `NOT IN` to prevent unexpected empty results due to NULL comparison semantics in SQL.

The ANY Operator

The `ANY` operator (synonymous with `SOME` in standard SQL) compares a value to **each value** returned by a subquery and returns `TRUE` if the comparison is satisfied for **at least one** of them. It is used with comparison operators like `=`, `>`, `<`, `>=`, `<=`, and `<>`.

Syntax

```
SELECT column_list
FROM table1
WHERE column operator ANY (
    SELECT column
    FROM table2
    WHERE condition
);
```

operator can be: `=`, `>`, `<`, `>=`, `<=`, `<>`

Example: Employees Earning More Than At Least One Employee in Dept 5

```
SELECT emp_name, salary
FROM Employee
WHERE salary > ANY (
    SELECT salary
    FROM Employee
    WHERE dept_id = 5
);
```

This returns all employees whose salary is greater than the **minimum** salary in department 5. The condition is satisfied if the employee's salary exceeds *any single* salary in that department — effectively comparing against the smallest value.

Special Case: = ANY

`= ANY` is functionally equivalent to `IN`. Both test whether a value equals at least one value in the subquery result.

The ALL Operator

The `ALL` operator compares a value to **every value** returned by a subquery and returns `TRUE` only if the comparison is satisfied for **all** of them. Like `ANY`, it is used with standard comparison operators and provides a stricter condition.

Syntax

```
SELECT column_list
FROM table1
WHERE column operator ALL (
    SELECT column
    FROM table2
    WHERE condition
);
```

Example: Employees Earning More Than ALL Employees in Dept 5

```
SELECT emp_name, salary
FROM Employee
WHERE salary > ALL (
    SELECT salary
    FROM Employee
    WHERE dept_id = 5
);
```

This returns employees whose salary exceeds the **maximum** salary in department 5. The condition must hold against *every* salary in that department — effectively comparing against the largest value.

ANY vs. ALL with > Operator

> ANY

Value must exceed the **minimum** in the subquery. Easier to satisfy.

> ALL

Value must exceed the **maximum** in the subquery. Harder to satisfy.

Special Case: = ALL

`= ALL` requires the value to equal **every** value in the subquery — only possible if all subquery values are identical or there is exactly one value.

ANY vs. ALL: Detailed Comparison

ANY — "At Least One"

Returns `TRUE` if the condition holds for **one or more** values in the subquery result. Less restrictive — easier to match.

- `> ANY` → compares to **MIN** of subquery
- `< ANY` → compares to **MAX** of subquery
- `= ANY` → equivalent to `IN`
- `<> ANY` → not equal to at least one (often TRUE)

ALL — "Every Single One"

Returns `TRUE` only if the condition holds for **all** values in the subquery result. More restrictive — harder to match.

- `> ALL` → compares to **MAX** of subquery
- `< ALL` → compares to **MIN** of subquery
- `= ALL` → equal to every value (rare)
- `<> ALL` → equivalent to `NOT IN`

Quick Reference: `> ANY` = "greater than the smallest," `> ALL` = "greater than the largest." `< ANY` = "less than the largest," `< ALL` = "less than the smallest."

Practical Examples: Putting It All Together

Below are four practical examples demonstrating each operator in realistic database scenarios. These illustrate how nested queries with `IN`, `NOT IN`, `ANY`, and `ALL` solve common data retrieval problems.

1

IN — Products in Active Categories

```
SELECT prod_name, price
FROM Product
WHERE cat_id IN (
   SELECT cat_id
   FROM Category
   WHERE status = 'Active'
);
```

Returns all products belonging to categories currently marked as active.

2

NOT IN — Students Not Enrolled in Any Course

```
SELECT student_name
FROM Student
WHERE student_id NOT IN (
   SELECT student_id
   FROM Enrollment
   WHERE student_id IS NOT NULL
);
```

Finds students who have no enrollment records — i.e., not registered for any course.

3

ANY — Projects Over Budget Compared to Some Dept

```
SELECT project_name, budget
FROM Project
WHERE budget > ANY (
   SELECT avg_budget
   FROM Department
);
```

Returns projects whose budget exceeds at least one department's average budget (i.e., above the minimum average).

4

ALL — Suppliers With Prices Below All Competitors

```
SELECT supplier_name
FROM Supplier
WHERE unit_price < ALL (
   SELECT unit_price
   FROM Competitor
);
```

Returns suppliers whose price is lower than every competitor's price — i.e., the cheapest overall (below the minimum competitor price).

Key Takeaways & Summary

Nested queries with `IN`, `NOT IN`, `ANY`, and `ALL` are powerful tools for expressing complex filtering logic in SQL. Understanding when and how to use each operator is essential for writing correct and efficient queries.

IN

Tests membership — returns rows where value **exists** in subquery. Safe with NULLs. Equivalent to `= ANY`.

NOT IN

Tests exclusion — returns rows where value is **absent** from subquery. **Beware of NULLs** — always filter them. Equivalent to `<> ALL`.

ANY

Tests against **at least one** value. Less restrictive. `> ANY` compares to MIN; `< ANY` compares to MAX.

ALL

Tests against **every** value. More restrictive. `> ALL` compares to MAX; `< ALL` compares to MIN.

Best Practices

- Always test subqueries independently before nesting
- Add `IS NOT NULL` guards in `NOT IN` subqueries
- Use `IN` instead of `= ANY` for clarity
- Consider performance — some databases optimize joins better
- Use `EXISTS` as an alternative for large datasets

Operator Equivalence Reference

Operator	Equivalent To
`= ANY`	`IN`
`<> ALL`	`NOT IN`
`> ANY`	Compare to MIN
`> ALL`	Compare to MAX
`< ANY`	Compare to MAX
`< ALL`	Compare to MIN